resolve w/ linked list

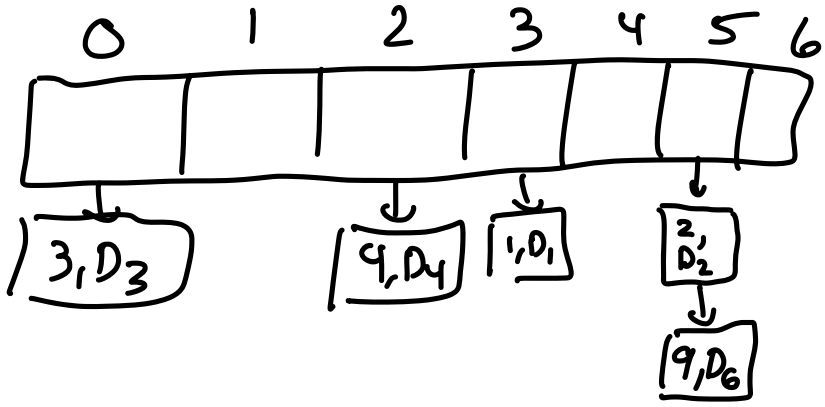
Problem 1. Let H be an initially empty hash table of size 7 implemented using chained hashing. Assume that the hash function for the hash table is

$$h(K) = (2K + 1) \bmod 7$$

Perform the following sequence of operations in order.

1: `HashTable.Insert`$(2, D_2)$
2: `HashTable.Insert`$(4, D_4)$
3: `HashTable.Insert`$(1, D_1)$
4: `HashTable.Insert`$(3, D_3)$
5: `HashTable.Insert`$(9, D_6)$

and draw the resulting hash table.



$h(2) = (2(2) + 1) \bmod 7 = 5$

$h(4) = (2(4) + 1) \bmod 7 = 2$

$h(1) = (2(1) + 1) \bmod 7 = 3$

$h(3) = (2(3) + 1) \bmod 7 = 0$

$h(9) = (2(9) + 1) \bmod 7 = 5$

Problem 2. Let H be a hash table of size 11 with the hash function

$$h(K, j) = (2K + 3j) \bmod 11$$

implemented using **open address hashing**. Any locations highlighted in **red** are occupied.

	0	1	2	3	4	5	6	7	8	9	10
H				$7, D_7$					$1, D_1$		

Consider the following sequence of operations

`Insert`$(1, D_1)$

`Insert`$(7, D_7)$

Draw the hash table after all of the above operations have been executed showing which elements are in which locations of the hash table and how they are stored. Show your work!

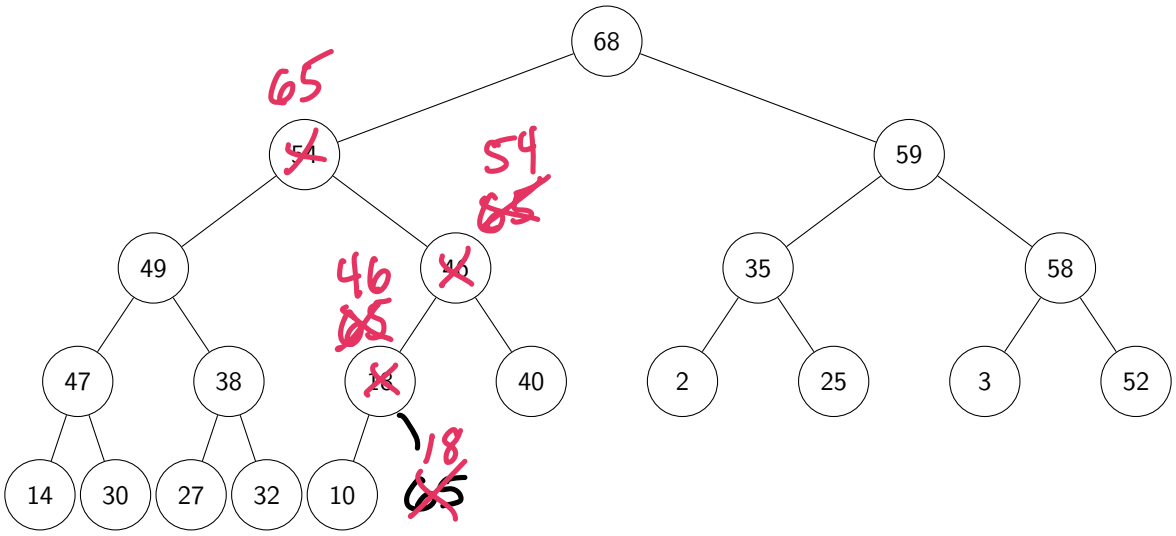
$h(1,0) = (2(1) + 3(0)) \bmod 11 = 2$ ✗

$h(1,1) = (2(1) + 3(1)) \bmod 11 = 5$ ✗

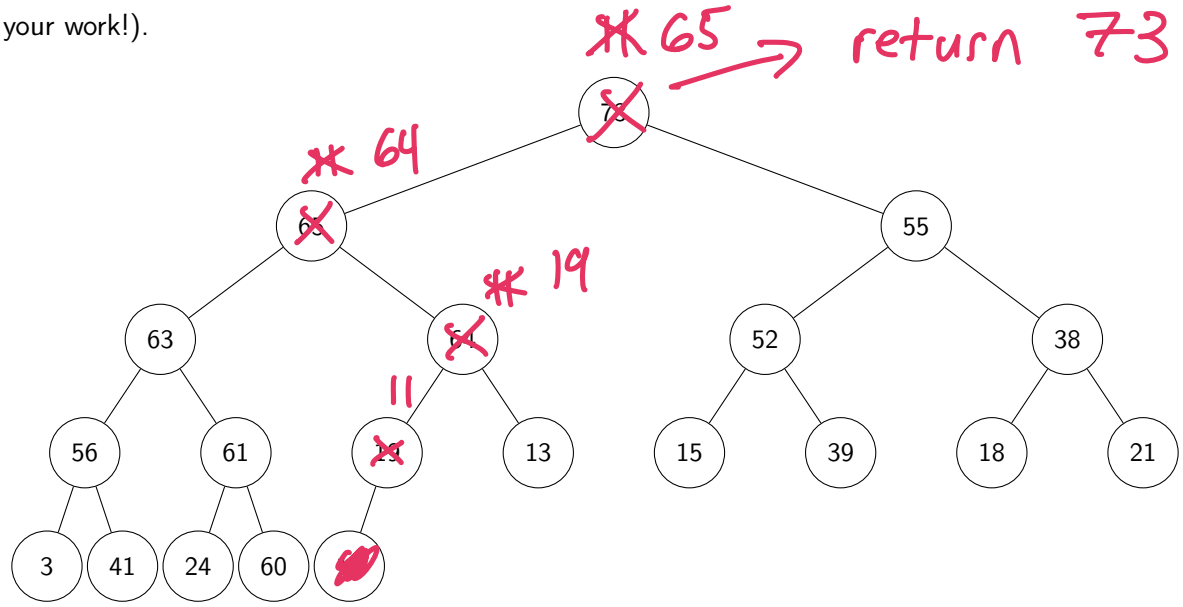
$h(1,2) = (2(1) + 3(2)) \bmod 11 = 8$ ✓

$h(7,0) = (2(7) + 3(0)) \bmod 11 = 3$ ✓

Problem 3. Apply the `Insert(65)` operation to the following max heap and draw the resulting binary tree (show your work!).



Problem 4. Apply the `ExtractMax()` operation to the following max heap and draw the resulting binary tree (show your work!).



Problem 5. Consider the algorithm given below

```
function Func(A[],n)
    HashTable.Init()
    x = 0                                   s=1
    for i = 1 to n do
        if HashTable.Member(A[i]) then      c
            HashTable.Add(A[i], 1)
        else
            HashTable.Insert(A[i],10)
        end if                              cs
    end for
    x = 0                                   s=n
    for i = 1 to n do
        if HashTable.Member(A[i]) then      c
            x = x + 1
        end if                              cs
    end for
    return x                                s=n
end function
```

$\sum_{i=1}^{n} c = cn$

$\sum_{i=1}^{n} c = cn$

Assume the Hash Table is implemented using Open Address Hashing and that the size of the Hash Table is at least $2n$. Determine the best, expected, and worst case running time.

BC/EC: All methods take $\Theta(1)$ time.

$\Theta(n)$

WC: All methods take $\Theta(s)$ time.

We always insert

$$W(n) = \sum_{s=1}^{n} cs + \sum_{i=1}^{n} \underbrace{cs}_{cn} = c\sum_{s=1}^{n} s + \sum_{i=1}^{n} cn$$

$$= c\frac{n(n+1)}{2} + n \cdot cn$$

$W(n) \in \Theta(n^2)$.

Problem 6. Consider table expansion where we increase the size of the table by a factor of 2 whenever the table is full. (For instance, a table with size 2 is expanded to size 4. A table with size 16 is expanded to size 32.) Each insertion without table expansion takes $\Theta(\log(s))$ time where s is the number of elements currently in the table. Table expansion takes ck^2 time where k is the size of the new table. What is the run time to perform n insertions? What is the Amortized run time of the n insertions?

$T(n)$ = total time for n insertions $\qquad AT(n) = \frac{\text{total}}{\#}$

$T(n) = [\text{time for inserts}] + [\text{time for expansions}]$

# elements	cost
1	$\log(1)$
2	$\log(2)$
3	$\log(3)$
$\vdots$	$\vdots$
	j
n	$\log(n)$

old	new	cost
1	2	$c(2^2)$
2	4	$c(4^2)$
4	8	$c(8)^2$
$\vdots$	$\vdots$	
$\frac{n}{2}$	n	$c(n^2)$

$$T(n) = \sum_{s=1}^{n} \log(s) + \sum_{k=1}^{\ell} c(2^k)^2$$

$$c(2^\ell)^2 = cn^2$$
$$4^\ell = n^2$$
$$\ell = \log_4(n^2)$$

$$T(n) = \underbrace{\sum_{s=1}^{n} \log(s)}_{\substack{\le \sum_{s=1}^{n} \log(n) \\ n\log(n)}} + \underbrace{\sum_{k=1}^{\log_4(n^2)} c\, 4^k}_{\Theta\left(4^{\log_4(n^2)}\right)}$$

$r > 1$

$\Theta(n^2)$

$$\sum_{i=0}^{n} r^i = \Theta(r^n) = \Theta(\text{LARGEST TERM})$$

Since $n^2 > n\log(n)$, it is dom. term.

$T(n) \in \Theta(n^2)$

$AT(n) = n^2/n$, $AT(n) \in \Theta(n)$.

If you have $\sum_{s=X-a}^{X}$

Problem 7. Consider the following problem:

```
function Func(A[],n)
    P.Init()
    for i = 1 to n^2 do
        k = Random(n)
        for j = 1 to k do
            P.Insert(A[i] · A[j])
        end for
    end for
    for i = 1 to n log(n) do
        x = P.ExtractMax()
        Print x
    end for
end function
```

$s=1$

$s=n^3$

$s=n^3-n\log(n)$

n

DO NOT CHANGE BOUNDS WHEN BOUNDING BELOW!

Assume that s is the number of elements currently in the Priority Queue. Analyze the **WORST CASE** running time of the given algorithm assuming that P is a Max-Heap.

WC occurs when $K=n$.

$$T(n) = \sum_{s=1}^{n^3} c\log(s) + \sum_{s=n^3-n\log(n)}^{n^3} c\log(s)$$

UB:

$$T(n) \le \sum_{s=1}^{n^3} c\log(n^3) + \sum_{s=n^3-n\log(n)}^{n^3} c\log(n^3)$$

$$T(n) \le \underbrace{n^3 \cdot c\log(n^3)}_{n^3\log(n)} + \underbrace{\underbrace{(n^3-(n^3-n\log(n))+1)}_{n\log(n)+1} c\log(n^3)}_{n\log(n)^2}$$

$O(n^3\log(n))$

LB:

$$T(n) \ge \sum_{s=\frac{n^3}{2}}^{n^3} c\log\left(\frac{n^3}{2}\right) + \sum_{s=n^3-n\log(n)}^{n^3} c\log(n^3-n\log(n))$$

$$T(n) \ge \left(n^3-\frac{n^3}{2}+1\right) c\log\left(\frac{n^3}{2}\right) + (n^3-(n^3-n\log(n))+1)\, c\log(n^3-n\log(n))$$

$\Omega(n^3\log(n))$

Note: could instead drop the 2nd sum in the LB.

Thus, $T(n) \in \Theta(n^3\log(n))$

- Validity of DS + operations Heaps 1.1, 1.4
- "Bad" DS. // "good" DS Hashing 1.2
- when to use a DS.